دانشگاه علم و صنعت ایران

دانشکده مهندسی کامپیوتر

آزمایشگاه داده‌کاوی

پردازش زبان طبیعی پیشرفته

نیم‌سال اول سال تحصیلی ۱۴۰۶–۱۴۰۵

(نیم‌سال ۴۰۵۱)

پروژه درس پردازش زبان طبیعی پیشرفته

پرسش و پاسخ چندمرحله‌ای با RAG و GraphRAG

مدرس درس: دکتر بهروز مینایی بیدگلی

مدیر دوره: دکتر نجمه ترابیان

سرپرست دستیاران آموزشی: پوریا صانعی

تاریخ انتشار پروژه: ۱۸ مهر

مهلت تحویل فاز ۲: ۲۱ آبان

شیوه انجام: انفرادی

دستیاران آموزشی مسئول پروژه (Project TAs)

محمد امین حائری (مسئول فاز ۱)
@RST_LSS

مریم مرتضوی نیک
@thisisnikk

پوریا صانعی (مسئول فاز ۲)
@PooryaSN

محمد خاکی
@khaki_mim

۱ قوانین و راهنمای تحویل پروژه

الزامات و قالب تحویل هر فاز

- **انفرادی بودن پروژه:** پروژه کاملاً **انفرادی** است و هر دانشجو باید تمامی پیاده‌سازی‌ها، آزمایش‌ها و نگارش گزارش‌ها را به‌طور مستقل انجام دهد.
- **فایل تحویلی:** یک فایل فشرده `P<N>_<StdID>.zip` (مثال: `P1_401234567.zip`) شامل:
  - گزارش فاز به‌صورت `PDF` (حداکثر ۳ صفحه؛ بخش ۱.۷)؛
  - یک پوشه برای هر خروجی، شامل `predictions.jsonl` و `run_config.yaml` (بخش ۵)؛
  - پیوند مخزن **خصوصی** GitHub شما به همراه **شناسه commit** متناظر با این تحویل (در صفحه اول گزارش). نحوه اعطای دسترسی خواندن به دستیار آموزشی در کانال درس اعلام می‌شود.
- **کد:** ساختار پروژه، زبان برنامه‌نویسی و چارچوب‌ها کاملاً به انتخاب شماست؛ کد نهایی شما در مخزن قرار می‌گیرد و ارزیابی در جلسه ارائه شفاهی بر اساس همان نسخه خواهد بود.

مهلت‌ها و سیاست تأخیر (Late Policy)

- **زمان‌بندی:** در این نسخه مهلت تحویل و بازه ارائه شفاهی فازهای ۱ و ۲ قطعی است (جدول بخش ۲). اطلاعات فازهای بعدی متعاقباً اعلام خواهد شد.
- **تأخیر مجاز ترم:** هر دانشجو در طول ترم در مجموع **۴۸ ساعت تأخیر مجاز** دارد.
- **جریمه تأخیر مازاد:** پس از اتمام این سهمیه، به ازای هر ساعت تأخیر **نیم نمره** از نمره ۱۰۰ نمره‌ای پروژه کسر می‌شود.
- **پاداش ارسال زودهنگام:** تحویل پیش از موعد با نمره بیش از **۸۰ درصد**، حداکثر **۲۴ ساعت** به سهمیه تأخیر مجاز می‌افزاید.
- **بسته شدن سامانه:** سامانه ارسال **۴ روز پس از هر مهلت** بسته خواهد شد.

شیوه ارزیابی (Grading)

- **کد و نتایج (حدود ۴۰ درصد):** صحت و کامل‌بودن خروجی‌ها، کیفیت گزارش، و راستی‌آزمایی چند پیش‌بینی توسط دستیار آموزشی (بررسی اینکه آیا پاسخ‌ها و قطعه‌متن‌های بازیابی‌شده واقعاً خروجی سامانه شما هستند یا خیر).
- **ارائه شفاهی (حدود ۶۰ درصد):** مؤلفه اصلی نمره‌دهی؛ میزان تسلط شما بر کد، سازوکارهای اصلی و نتایج به‌دست‌آمده (بخش ۲.۷).
- **وزن فازها:** وزن نهایی فازهای بعدی پس از انتشار برنامه کامل پروژه اعلام خواهد شد.
- **عدم رقابت و رتبه‌بندی:** هیچ نمره هدف مشترک یا رتبه‌بندی رقابتی میان دانشجویان وجود ندارد؛ هر دانشجو سامانه‌های خود را صرفاً با خطوط پایه **خودش** مقایسه می‌کند.

صداقت دانشگاهی و استفاده از دستیارهای هوش مصنوعی

تمامی فایل‌های ارسالی باید حاصل کار مستقل شما باشند. شباهت‌سنجی کد و گزارش به‌صورت خودکار انجام می‌پذیرد. در صورت مشاهده هرگونه کپی‌برداری، برای **تمامی طرفین** دست‌کم **۵۰ درصد جریمه نمره** لحاظ خواهد شد. استفاده از ابزارهای هوش مصنوعی برای کدنویسی **مجاز** است، مشروط بر آنکه بر کد خود تسلط کامل داشته باشید و نحوه بهره‌گیری از آن‌ها را در گزارش قید کنید.

نحوه ارتباط و پاسخ به سؤالات

سؤالات و ابهامات هر فاز را در پیام‌رسان تلگرام از دستیار آموزشی مسئول همان فاز (جدول بخش ۲) بپرسید.

۲ تقویم مهلت‌ها و دستیاران مسئول

زمان‌بندی مهلت‌های تحویل و بازه‌های ارائه شفاهی فازهای مختلف پروژه در جدول زیر آمده است. دستیاران مسئول فازهای ۳ تا ۵ و جزئیات تکمیلی فاز ۵ متعاقباً اعلام خواهد شد.

جدول مهلت‌های تحویل و ارائه‌های شفاهی

فاز	موضوع	مهلت تحویل	بازه ارائه شفاهی	دستیار مسئول
انتشار	انتشار صورت پروژه و بسته داده	۱۸ مهر	—	—
فاز ۱	سنجش و بازیابی متراکم	پنج‌شنبه ۳۰ مهر ۱۴۰۵	همراه با فاز دوم	محمد امین حائری
فاز ۲	بازیابی ترکیبی و پاسخ مستند	پنج‌شنبه ۲۱ آبان ۱۴۰۵	یکشنبه ۲۴ تا پنج‌شنبه ۲۸ آبان	پوریا صانعی
فاز ۳	گراف دانش	پنج‌شنبه ۱۲ آذر ۱۴۰۵	یکشنبه ۱۵ تا پنج‌شنبه ۱۹ آذر	بعداً اعلام می‌شود
فاز ۴	کنترل چندمرحله‌ای و مقایسه نهایی	پنج‌شنبه ۳ دی ۱۴۰۵	یکشنبه ۶ تا پنج‌شنبه ۱۰ دی	بعداً اعلام می‌شود
فاز ۵	توضیحات تکمیلی در آینده منتشر می‌شود	پنج‌شنبه ۱ بهمن ۱۴۰۵	یکشنبه ۴ تا سه‌شنبه ۷ بهمن	بعداً اعلام می‌شود

نکات زمان‌بندی

- زمان دقیق جلسه ارائه شفاهی هر فاز را دستیار آموزشی مسئول آن اعلام می‌کند؛ ارائه شفاهی فاز ۱ همراه با فاز ۲ و در بازه ۲۴ تا ۲۸ آبان برگزار خواهد شد.
- دستیاران مسئول فازهای ۳ تا ۵ و جزئیات تکمیلی فاز ۵ متعاقباً از طریق کانال رسمی درس اعلام خواهد شد.

فهرست مطالب

۱ قوانین و راهنمای تحویل پروژه ۲

۲ تقویم مهلت‌ها و دستیاران مسئول ۳

۳ مقدمه و اهداف ۶

۱.۳ اهداف یادگیری ۷

۲.۳ آنچه ثابت است و آنچه آزاد است ۷

۴ داده‌ها: بسته درس course pack v1 ۸

۱.۴ فایل‌ها و ساختار داده ۸

۲.۴ مجموعه‌های dev و hidden ۹

۳.۴ انواع پرسش ۹

۵ فایل اجرا، پیکربندی اجرا و امتیازدهی ۱۰

۱.۵ قالب predictions.jsonl ۱۰

۲.۵ پیکربندی اجرا: run_config.yaml ۱۱

۳.۵ اسکریپت امتیازدهی score.py ۱۱

۶ مدل‌ها و منابع پردازشی ۱۲

۱.۶ بسترهای اجرای مدل ۱۳

۲.۶ خودآزمایی صحت مدل (پیشنهادی) ۱۳

۷ گزارش و ارائه شفاهی ۱۴

۱.۷ ساختار گزارش هر فاز ۱۴

۲.۷ جلسه ارائه شفاهی ۱۴

۸ فاز ۱: سنجش و بازیابی متراکم ۱۵

۱.۸ سازوکار اصلی الف: معیارهای ارزیابی بازیابی ۱۵

۲.۸ سازوکار اصلی ب: بازیابی متراکم دقیق با ماسک اسناد مستثنی (Document Mask) ۱۵

۳.۸ سامانه‌های این فاز ۱۶

۹ فاز ۲: بازیابی ترکیبی (هایبرید) و پاسخ‌های مستند ۱۷

۱.۹ سازوکار اصلی الف: الگوریتم BM25 و ادغام رتبه‌ها با RRF ۱۷

۲.۹ سازوکار اصلی ب: بسته‌بندی کانتکست، استناددهی و امتناع از پاسخ ۱۸
۳.۹ آزمایش‌های تجربی این فاز ۱۸

آ پیوست اختیاری: امتیاز مهندسی نرم‌افزار و ردیابی آزمایش‌ها ۱۹
آ.۱ پیکربندی ساختاریافته، آزمون خودکار و یکپارچه‌سازی مداوم ۱۹
آ.۲ ردیابی آزمایش‌ها با MLflow یا W&B ۱۹

۳ مقدمه و اهداف

خلاصه پروژه

در این پروژه، هر دانشجو چندین سامانه پرسش و پاسخ مبتنی بر بازیابی متنی و ساختاریافته (**RAG** و **GraphRAG**) را برای **پرسش‌های چندمرحله‌ای** پیاده‌سازی کرده و آن‌ها را در شرایطی یکسان و منصفانه با یکدیگر مقایسه می‌کند. پاسخ هیچ‌یک از پرسش‌ها در یک قطعه‌متن (passage) منفرد قرار ندارد؛ بلکه برای رسیدن به پاسخ نهایی، باید دو یا چهار قطعه‌متن مرتبط از پیکره ثابت مقاله‌های ویکی‌پدیای انگلیسی بازیابی شده و با یکدیگر ترکیب شوند.

پروژه در قالب **چهار فاز** متوالی (هر فاز با مهلت تحویل مستقل) پیش می‌رود: سنجش و بازیابی متراکم؛ بازیابی ترکیبی و پاسخ مستند؛ گراف دانش و GraphRAG؛ کنترل چندمرحله‌ای و مقایسه نهایی. شما در انتخاب کتابخانه‌ها و چارچوب‌های توسعه کاملاً آزاد هستید (LlamaIndex، LangChain، FAISS، sentence-transformers، rank_bm25، Ollama، vLLM و غیره). **هیچ مخزن کد اولیه، دستور خط فرمان اجباری یا ساختار کد از پیش تحمیل‌شده‌ای وجود ندارد.**

تنها مؤلفه‌های زیر در طول پروژه ثابت و یکسان هستند: بسته داده درس (course pack v1)، قالب فایل خروجی اجرا (`predictions.jsonl`)، اسکریپت ارزیابی و امتیازدهی تیم درس (`score.py`)، فایل پیکربندی `run_config.yaml` به ازای هر خروجی، و در هر فاز **دو «سازوکار اصلی»** (core mechanism) که باید خودتان از ابتدا پیاده‌سازی کنید و در جلسه ارائه شفاهی به تشریح کد آن بپردازید.

مدل‌های زبانی بزرگ معمولاً به پرسش‌های ساده پیرامون موجودیت‌های سرشناس بر اساس حافظه پارامتریک خود پاسخ می‌دهند؛ اما هنگامی که پاسخ نیازمند ترکیب چندین واقعیت پراکنده باشد یا اطلاعات مد نظر کمتر شناخته‌شده باشند، این مدل‌ها مستعد خطا و تولید پاسخ‌های نادرست اما با اطمینان کاذب هستند. سامانه‌های **RAG** پیش از تولید پاسخ، ابتدا **قطعه‌متن‌های** مرتبط را از یک پیکره متنی بازیابی کرده و پاسخ را مستند به شواهد استخراج‌شده تولید می‌کنند. در پرسش‌های **چندمرحله‌ای** (**multi-hop**)، یک مرحله بازیابی ساده معمولاً پاسخگو نیست؛ به عنوان مثال، برای پاسخ به پرسش «کارگردان کدام فیلم دیرتر به دنیا آمده است؟»، سامانه باید ابتدا مقالات هر دو فیلم را بازیابی کند، نام کارگردانان را بیابد و سپس با بازیابی قطعه‌متن‌های مربوط به تاریخ تولد هر دو کارگردان، پاسخ نهایی را استخراج نماید.

در چنین سناریوهایی، بازیابی متنی سنتی به تنهایی با چالش اساسی روبه‌رو می‌شود؛ زیرا نام موجودیت‌های واسط (مانند کارگردان‌ها) در صورت پرسش غایب است و بازیاب متنی در گام نخست نمی‌داند به دنبال چه اسنادی بگردد. اینجاست که رویکرد **GraphRAG** با استخراج روابط ساختاریافته، تشکیل گراف دانش و پیمایش زنجیره‌ای میان گره‌ها وارد عمل می‌شود تا اتصال میان شواهد پراکنده را برقرار سازد. تحلیل تجربی اثر **GraphRAG** و سنجش نقش آن در پر کردن شکاف استدلال چندمرحله‌ای، یکی از محورهای بنیادین این پروژه است.

این پروژه یک **مطالعه تجربی و پژوهشی** است، نه یک مسابقه رقابتی. در فازهای نخست تا چهارم، زنجیره‌ای از سامانه‌های مختلف را از خطوط پایه تا رویکردهای گرافی و چندمرحله‌ای پیاده‌سازی کرده و در چارچوبی یکسان و کنترل‌شده ارزیابی می‌کنید تا اثر واقعی هر معماری بازیابی در کیفیت پاسخ‌ها مشخص گردد.

۱.۳ اهداف یادگیری

در پایان این پروژه، دانشجویان قادر خواهند بود:

۱. معیارهای ارزیابی بازیابی اطلاعات و تولید پاسخ را پیاده‌سازی و تحلیل کنند و تفاوت‌های عملکردی را همراه با **فاصله اطمینان** گزارش نمایند؛
۲. یک بازیاب متراکم دقیق (Exact Dense Retriever) و بازیاب ترکیبی پیاده‌سازی کرده و اثر بازیابی متنی را بر کیفیت پاسخ نهایی بسنجند؛
۳. اصول، معماری و سازوکارهای رویکرد **GraphRAG** (شامل استخراج سه‌تایی‌های دانش با مدل‌های زبانی، اعتبارسنجی گزاره‌ها، و پیمایش چندمرحله‌ای گراف همراه با ردیابی منشأ اسناد) را پیاده‌سازی کنند؛
۴. **تأثیر و ارزش افزوده رویکرد GraphRAG** را در اتصال موجودیت‌های واسط و بهبود پوشش شواهد در پرسش‌های چندمرحله‌ای (multi-hop) به‌صورت کمی و کیفی ارزیابی و تحلیل نمایند؛
۵. برای هر پرسش، قطعه‌متن‌های بازیابی‌شده را رتبه‌بندی کرده و معیارهای استاندارد بازیابی اطلاعات را محاسبه و تفسیر کنند؛
۶. سامانه‌های متنوع (Dense RAG، Hybrid RAG، GraphRAG و Decomposed RAG) را به‌صورت **جفتی** (paired) و در شرایطی کنترل‌شده و منصفانه با یکدیگر مقایسه نمایند؛
۷. پیرامون تصمیمات طراحی سامانه، یافته‌های تجربی و محدودیت‌های روش خود به‌صورت مستدل و علمی در ارائه شفاهی توضیح دهند.

۲.۳ آنچه ثابت است و آنچه آزاد است

ثابت (برای همه یکسان)	آزاد (انتخاب شما)
بسته داده course pack v1 و تفکیک hidden/dev	زبان برنامه‌نویسی، کتابخانه‌ها و چارچوب‌ها
قالب فایل اجرا `predictions.jsonl`	معماری کد و ساختار پیاده‌سازی
اسکریپت امتیازدهی `score.py`	مهندسی پرامپت، تنظیمات بازیابی، واحد تکه‌بندی متن (chunking)
فایل `run_config.yaml` برای هر خروجی	انتخاب مدل زبانی، مدل تعبیه و بازرتبه‌بندی‌کننده (با رعایت قیود بخش ۶)
دو سازوکار اصلی دست‌نویس در فاز ۱	بستر اجرا: رایانه شخصی، Kaggle، سرور شخصی

آنچه در اختیارتان قرار می‌گیرد

- `course_pack_v1/` : پیکره حاوی ۳۸۸۲ قطعه‌متن، ۳۰۰ پرسش ارزیابی (dev) به همراه برچسب، ۳۰۰ پرسش آزمون پنهان (hidden) بدون برچسب.
- `predictions_format.md` : مستندات مشخصات و قالب فایل اجرا.
- `score.py` : اسکریپت اعتبارسنجی قالب فایل و محاسبه خودکار معیارها همراه با فاصله اطمینان bootstrap.
- `example_run_config.yaml` و `example_predict.py` : نمونه فایل پیکربندی اجرا و یک اسکریپت پایه حدوداً ۳۰ خطی جهت تولید فایل اجرای معتبر ساختاری.

منظور از «سازوکار اصلی» (Core Mechanism) چیست؟

در هر فاز، دو بخش کلیدی و الگوریتمی سامانه باید **توسط خود شما و از ابتدا** کدنویسی شوند، حتی اگر مابقی سامانه با بهره‌گیری از چارچوب‌های آماده پیاده‌سازی شده باشد (برای نمونه، ممکن است بازیابی متراکم با LlamaIndex پیاده‌سازی شود، اما محاسبه معیار nDCG باید پیاده‌سازی دست‌نویس خودتان باشد). این بخش‌ها معمولاً کدهایی در حد چند ده خط هستند. هیچ آزمون خودکاری بر روی آن‌ها انجام نمی‌شود؛ بلکه **صرفاً در جلسه ارائه شفاهی** توسط دستیار آموزشی بررسی می‌شوند: شما کد را باز کرده، منطق پیاده‌سازی را گام‌به‌گام توضیح می‌دهید و به پرسش‌های مرتبط پاسخ خواهید داد.

۴ داده‌ها: بسته درس course pack v1

بسته داده شامل یک پیکره متنی ثابت از **قطعه‌متن‌ها** (passages) و دو مجموعه پرسش است. هر قطعه‌متن برشی از مقاله‌های ویکی‌پدیای انگلیسی است (غالباً بند آغازین مقاله‌ای پیرامون یک فیلم، شخص، مکان یا اثر) و عنوان آن بر عنوان مقاله منطبق است. تمامی پرسش‌ها به زبان انگلیسی طراحی شده و پاسخ‌ها کوتاه هستند: یک موجودیت، تاریخ، مکان، عدد یا پاسخ‌های دودویی yes/no.

۱.۴ فایل‌ها و ساختار داده

```
course_pack_v1/
  corpus.jsonl            3,882 passages  {"doc_id", "title", "text", "sentences": [...]}
  questions_dev.jsonl     300 questions   {"qid", "question", "type"}
  labels_dev.jsonl        their labels    {"qid", "answer", "aliases", "gold_doc_ids",
                                           "gold_sentences", "evidence_triples", "type"}
  questions_hidden.jsonl  300 questions without labels (the TAs score them)
  kg_subset_passages.txt  200 doc ids for the Phase 3 extraction exercise
  README.txt
```

- **پیکره** (`corpus.jsonl`): هر سطر بیانگر یک قطعه‌متن با شناسه‌ای مشخص مانند `p00001`، عنوان، متن کامل و فهرست جملات تقطیع‌شده آن (`sentences`؛ به‌طور میانگین ۳ تا ۴ جمله) است. طول قطعه‌متن‌ها تنوع بالایی دارد: اکثریت آن‌ها کوتاه هستند، اما حدود ۸ درصد از آن‌ها طولی بیش از ۲۵۶ توکن دارند (میانه طول حدود ۶۰ توکن و حداکثر طول در حدود ۱۲۰۰ توکن با توکن‌ساز bge-small).
- **پرسش‌ها** (`questions_<split>.jsonl`): شامل شناسه (`qid`)، متن پرسش (`question`) و دسته‌بندی نوع پرسش (`type`) است. فیلد نوع پرسش برای هر دو مجموعه در دسترس بوده و استفاده از آن در خط لوله مجاز است.
- **برچسب‌ها** (`labels_dev.jsonl`، صرفاً برای مجموعه dev):
  - `answer` و `aliases`: پاسخ صحیح و صورت‌های نگارشی مترادف آن (هر یک به عنوان پاسخ صحیح پذیرفته می‌شود)؛
  - `gold_doc_ids`: شناسه‌های قطعه‌متن‌های طلایی که اجتماع آن‌ها برای پاسخ‌گویی به پرسش ضروری و کافی است؛
  - `gold_sentences`: جفت‌های [doc_id, sentence_index] که جملات حاوی شواهد اصلی را معین می‌کنند؛
  - `evidence_triples`: سه‌تایی‌های دانش به‌صورت [subject, relation, object] که زنجیره استدلال چندمرحله‌ای را نشان می‌دهند.

یک نمونه واقعی از داده‌های مجموعه dev (پرسش و برچسب‌های متناظر):

```
{"qid": "q0002", "question": "Which film has the director who was born later, The Third Side Of
 The River or Escape From L.A.?", "type": "bridge_comparison"}
{"qid": "q0002", "answer": "The Third Side Of The River",
 "aliases": ["The Third Side of the River", "Third Side of the River"],
 "gold_doc_ids": ["p03489", "p01066", "p00620", "p01750"],
 "gold_sentences": [["p03489", 0], ["p01066", 0], ["p00620", 0], ["p01750", 0]],
 "evidence_triples": [["The Third Side of the River", "director", "Celina Murga"],
                      ["Escape from L.A.", "director", "John Carpenter"],
                      ["Celina Murga", "date of birth", "April 6, 1973"],
                      ["John Carpenter", "date of birth", "January 16, 1948"]], "type": "bridge_comparison"}
```

۲.۴ مجموعه‌های dev و hidden

- **dev (۳۰۰ پرسش، همراه با برچسب):** برای توسعه، اشکال‌زدایی و **تمامی مقایسه‌ها و تحلیل‌های درون گزارش**. ارزیابی این بخش با اجرای اسکریپت `score.py` بر عهده خود شماست.
- **hidden (۳۰۰ پرسش، بدون برچسب):** برای تحویل نهایی فازها؛ خروجی پیش‌بینی‌های مدل خود را روی این مجموعه ارسال می‌کنید و نمره‌دهی نهایی آن توسط دستیاران آموزشی انجام می‌شود. شما صرفاً مجاز به اعتبارسنجی ساختار فایل اجرا روی این مجموعه هستید.

از آنجا که مجموعه dev تنها داده‌ای است که به برچسب‌های آن دسترسی دارید، احتمال بیش‌برازش و تنظیم افراطی روی آن (overfitting on dev) وجود دارد. تعداد آزمایش‌ها را منطقی مدیریت کنید، انتخاب‌های مدل‌سازی را مستدل انجام داده و در گزارش بازتاب دهید؛ مقایسه عملکرد روی dev و hidden در انتهای ترم، میزان تعمیم‌پذیری سامانه‌های شما را نمایان می‌سازد.

۳.۴ انواع پرسش

هر پرسش به یکی از چهار دسته ساختاری زیر تعلق دارد. مثال‌ها فرضی بوده و صرفاً جهت تبیین ساختار استدلال آورده شده‌اند.

نوع	تعداد در dev	تعداد قطعه‌متن‌های طلایی	ساختار و مثال ساختگی
comparison	۷۳	۲	مقایسه یک ویژگی مشترک میان دو موجودیت. «آیا فیلم‌های Blue Harbor و Silent Orchard در یک کشور تولید شده‌اند؟» (یک قطعه‌متن برای هر فیلم؛ پاسخ عموماً no/yes یا نام یکی از موجودیت‌ها است).
bridge_comparison	۶۶	۴	مقایسه از طریق موجودیت واسط. «کارگردان کدام فیلم زودتر متولد شده است، Blue Harbor یا Silent Orchard؟» (دو فیلم و دو کارگردان: نیازمند چهار قطعه‌متن).
compositional	۱۲۵	۲	استدلال زنجیره‌ای دو مرحله‌ای. «همسر نویسنده رمان The Glass Lantern در کدام شهر به دنیا آمده است؟» (مرحله اول: شناسایی نویسنده؛ مرحله دوم: ویژگی همسر او).
inference	۳۶	۲	استنتاج یک رابطه ترکیبی از پیوند دو رابطه پایه. «پدربزرگ مادری Anna Verhoeven کیست؟» (شناسایی مادر، سپس شناسایی پدرِ مادر).

چالش اصلی پروژه

در بازیابی ساده و تک‌مرحله‌ای، در پرسش‌های از نوع bridge_comparison به‌ندرت هر **چهار** قطعه‌متن طلایی در رتبه‌های برتر قرار می‌گیرند؛ زیرا نام کارگردان‌ها در صورت پرسش ذکر نشده و بازیاب در گام نخست هدفی برای یافتن آن‌ها ندارد. این محدودیت را در فاز ۱ با استفاده از معیار HitAll@k ارزیابی خواهید کرد.

۵ فایل اجرا، پیکربندی اجرا و امتیازدهی

منظور از «اجرا» (**run**)، خروجی حاصل از یک سامانه بر روی یکی از مجموعه‌های داده (dev یا hidden) است. در پیاده‌سازی سامانه خود آزاد هستید؛ اما خروجی نهایی آن لزوماً باید با ساختار زیر همخوانی داشته باشد تا توسط اسکریپت `score.py` و دستیاران آموزشی قابل خواندن و ارزیابی باشد.

۱.۵ قالب predictions.jsonl

هر سطر باید شامل یک شیء JSON معتبر برای یک پرسش باشد؛ به ازای هر `qid` از مجموعه دقیقاً یک سطر باید وجود داشته باشد و ثبت پرسش اضافی یا تکراری مجاز نیست.

فیلد	نوع	اجباری	معنا
`qid`	string	بله	شناسه پرسش برگرفته از `questions_<split>.jsonl`
`answer`	string	بله	پاسخ کوتاه نهایی (موجودیت، تاریخ، مکان، عدد یا no/yes)؛ در صورت امتناع: `""`
`retrieved`	list of strings	بله	فهرست شناسه‌های قطعه‌متن‌های `corpus.jsonl`، **مرتب‌شده بر اساس امتیاز (بهترین در ابتدا)** و بدون عضو تکراری؛ توصیه می‌شود حداقل ۱۰ شناسه درج شود (معیارها $k \in \{2, 5, 10\}$ را ارزیابی می‌کنند). برای سامانه بدون‌بازیابی مقدار آن `[]` است.
`cited`	list of strings	خیر	شناسه قطعه‌متن‌هایی که پاسخ تولیدی بر پایه آن‌ها مستند شده است
`abstained`	bool	خیر	مقدار `true` در صورتی که سامانه صریحاً از پاسخ دادن امتناع کرده باشد

```
{"qid": "q0007", "answer": "Steven Spielberg", "retrieved": ["p01234", "p00871", "p03002"],
 "cited": ["p01234"], "abstained": false}
```

درج هیچ فیلد اضافی دیگری مجاز نیست. در فاز ۱، شناسه‌های قطعه‌متن‌ها را بدون عضو تکراری و بر اساس رتبه شباهت درج نمایید.

۲.۵ پیکربندی اجرا: run_config.yaml

هر فایل اجرا باید در **پوشه اختصاصی خود** و در کنار یک فایل `run_config.yaml` قرار گیرد. این فایل پیکربندی مستقیماً توسط دستیاران آموزشی بررسی می‌شود تا مشخصات دقیق مدل‌ها، فراپارامترها و دستور خط فرمان تولید اجرا مشخص باشد. نمونه فایل (`example_run_config.yaml`):

```
system_name: dense_rag              # your own name for this system
phase: 1
split: hidden                       # dev or hidden
description: "bge-small dense retrieval, top-5 passages, strict answer prompt"
models:                             # keep identical across the Phase 1 systems
  llm: {name: Qwen/Qwen3-8B, where: "Kaggle T4, vLLM, 4-bit AWQ", temperature: 0}
  embedder: BAAI/bge-small-en-v1.5
  reranker: null
settings:                           # anything that defines this run
  retrieval: {unit: passage, k: 5}
  max_context_tokens: 1800
  prompt_file: prompts/strict.txt
command: "python src/run.py --config configs/dense.yaml --split hidden"
date: 2026-11-10
```

بخش `settings` آزاد است: هر مؤلفه‌ای که شرایط اجرای شما را معین می‌کند در آن قید نمایید. یک ساختار پیشنهادی (و نه اجباری) برای سازمان‌دهی فایل تحویلی:

```
P1_401234567.zip
  report.pdf
  runs/dev/closed_book/{predictions.jsonl, run_config.yaml}
  runs/dev/oracle/{predictions.jsonl, run_config.yaml}
  runs/dev/dense/{predictions.jsonl, run_config.yaml}
  runs/hidden/closed_book/{predictions.jsonl, run_config.yaml}
  runs/hidden/dense/{predictions.jsonl, run_config.yaml}
```

۳.۵ اسکریپت امتیازدهی score.py

اسکریپت `score.py` ارائه‌شده توسط تیم درس، صرفاً از کتابخانه استاندارد پایتون بهره می‌برد. این اسکریپت ابتدا فایل پیش‌بینی را **اعتبارسنجی ساختاری** می‌کند و در صورت بروز خطا، فهرست مغایرت‌ها را نمایش داده و متوقف می‌شود؛ در صورت صحت قالب، معیارهای ارزیابی را برای کل پرسش‌ها و نیز به تفکیک نوع پرسش، به همراه **فاصله اطمینان ۹۵ درصدی روش Bootstrap** محاسبه و چاپ می‌نماید.

```
# a valid (but useless) example run
python example_predict.py --pack course_pack_v1 --split dev --out runs/example/predictions.jsonl

# score a dev run
python score.py runs/dense/predictions.jsonl --pack course_pack_v1 --split dev

# paired difference on the same questions (this file minus the --compare file)
python score.py runs/hybrid/predictions.jsonl --pack course_pack_v1 --split dev \
       --compare runs/dense/predictions.jsonl
```

```
# check the format only (use this for your hidden-set files before submitting)
python score.py runs/hidden/dense/predictions.jsonl --pack course_pack_v1 --split hidden \
       --validate-only
```

معیارهای محاسبه‌شده. فرض کنید G مجموعه قطعه‌متن‌های طلایی و R_k مجموعه k قطعه‌متن نخستِ رتبه‌بندی‌شده در `retrieved` باشد:

- Recall@k برابر $|G \cap R_k|/|G|$ و HitAll@k برابر ۱ است هرگاه $G \subseteq R_k$ باشد (تمامی قطعه‌متن‌های طلایی در میان k نتیجه برتر حضور داشته باشند)، برای $k \in \{2, 5, 10\}$؛
- MRR: معکوس رتبه نخستین قطعه‌متن طلایی؛ تعریف nDCG@10 در فاز ۱ تشریح شده است؛
- EM و F1: به ترتیب تطابق کامل (Exact Match) و امتیاز F1 در سطح توکن، پس از نرمال‌سازی متن (تبدیل به حروف کوچک، حذف علائم نگارشی و حروف تعریف انگلیسی a/an/the) با در نظر گرفتن بیشینه بر روی `answer` و `aliases`. پرسش‌های no/yes به‌صورت دودویی ارزیابی می‌شوند. پاسخ خالی یا امتناع نمره صفر دریافت می‌کند؛
- cite_precision و cite_recall: در صورت پر کردن فیلد اختیاری `cited`؛ معیار abstained: درصد امتناع، در صورت ثبت فیلد اختیاری `abstained`.

> **مستندسازی ادعاهای مقایسه‌ای با فاصله اطمینان**
>
> تعداد ۳۰۰ پرسش حجم داده بزرگی نیست؛ بنابراین اختلاف‌های جزئی (مثلاً ۲ نمره) میان دو سامانه ممکن است ناشی از نوسانات تصادفی نمونه‌گیری باشد. هرگونه ادعای برتری نظیر «سامانه الف بهتر از سامانه ب عمل می‌کند» در گزارش باید مستند به **تفاضل جفتی و فاصله اطمینان آن** (خروجی پرچم `--compare`) باشد. چنانچه فاصله اطمینان عدد صفر را در بر گیرد، باید صریحاً قید نمایید: «تفاوت معنادار آماری مشاهده نشد».

۶ مدل‌ها و منابع پردازشی

در فاز ۱، یک مدل زبانی (LLM)، یک مدل تعبیه (embedding model) و در صورت تمایل یک بازرتبه‌بندی‌کننده (reranker) انتخاب کرده و مشخصات آن‌ها را در فایل `run_config.yaml` ثبت می‌نمایید. به منظور امکان مقایسه منصفانه و کنترل‌شده، مدل‌های پایه در هر سه سامانه این فاز باید کاملاً یکسان باشند.

> **قید مدل زبانی برای نتایج رسمی گزارش**
>
> برای **تمامی نتایج گزارش‌شده در بخش‌های اصلی** (شامل جداول، مقایسه‌ها، و خروجی‌ها بر روی مجموعه hidden)، مدل زبانی باید الزاما از میان مدل‌های **با وزن‌های آزاد** (**open-weight**) و با **حداکثر اندازه حدود ۱۴ میلیارد پارامتر (14B)** انتخاب شود؛ بهره‌گیری از هر روش کوانتایزیشن (quantization) مجاز است. آزمایش با مدل‌های بزرگ‌تر یا مدل‌های تجاری بسته (از طریق API) صرفاً در یک بخش الحاقی مجزا با عنوان «آزمایش‌های تکمیلی» مجاز خواهد بود.

علت اعمال این قید:

- **آشکار شدن نقش بازیابی:** مدل‌های زبانی بسیار بزرگ پاسخ بسیاری از این پرسش‌ها را صرفاً بر مبنای حافظه پارامتریک خود و بدون نیاز به زمینه می‌دانند؛ در چنین شرایطی، سامانه بدون‌بازیابی (**closed-book**) کیفیتی نزدیک به RAG ارائه می‌دهد و سهم بهبود حاصل از بازیابی به وضوح نمایان نمی‌شود.
- **عدالت و برابری ارزیابی:** تمامی دانشجویان با منابع پردازشی در دسترس و یکسان کار می‌کنند و کیفیت نتایج به پرداخت هزینه برای APIهای تجاری وابسته نخواهد بود.

- **امکان اجرا بر بسترهای محاسباتی رایگان:** مدل‌های تا سقف ۱۴ میلیارد پارامتر با کوانتایزیشن ۴ بیتی، بزرگ‌ترین مدل‌هایی هستند که امکان بارگذاری روی پردازنده‌های گرافیکی رایگان (مانند دو پردازنده T4 در Kaggle) یا رایانه‌های شخصی قدرتمند را دارند.

۱.۶ بسترهای اجرای مدل

اجرای مدل بر روی سخت‌افزار شخصی **پیشنهاد می‌شود اما اجباری نیست**:

- **رایانه شخصی:** با ابزارهایی چون Ollama، llama.cpp یا MLX (روی سیستم‌عامل مک) همراه با یک مدل ۴ تا ۸ میلیارد پارامتری کوانتایزشده.
- **پردازنده گرافیکی رایگان Kaggle:** با چارچوب‌های vLLM یا transformers؛ راهنمای جامع کار با Kaggle همراه این سند ارائه شده است.
- **سرویس‌های ارائه‌دهنده میزبانی مدل:** در صورت استفاده از ارائه‌دهندگان سرویس API برای فراخوانی یک مدل با وزن‌های آزاد، همان مدل و همان محدودیت سقف اندازه (حداکثر 14B) حاکم خواهد بود و باید مشخصات آن در `run_config.yaml` درج شود.

در تمامی اجراها دمای تولید (temperature) را روی مقدار صفر (یا بسیار نزدیک به صفر) تنظیم کرده و فایل پرامپت‌ها را در مخزن کد نگهداری کنید تا نتایج کاملاً بازتولیدپذیر باشند. بردارهای تعبیه پیکره را یک‌بار محاسبه کرده و در قالب فایل ذخیره نمایید؛ محاسبه بردار تعبیه ۳،۸۸۲ قطعه‌متن با یک مدل سبک، حتی روی پردازنده مرکزی (CPU) ظرف چند دقیقه به اتمام می‌رسد.

۲.۶ خودآزمایی صحت مدل (پیشنهادی)

پیش از صرف زمان و اجرای آزمایش‌های گسترده، این آزمون اولیه را انجام دهید (انجام آن اختیاری است، اما گزارش نتایج آن در فاز ۱ ارزشمند خواهد بود):

خودآزمایی سامانه با قطعه‌متن‌های طلایی (Sanity Check)

۱. تعداد ۵۰ پرسش از مجموعه dev انتخاب کرده و به ازای هر یک، **قطعه‌متن‌های طلایی** (برگرفته از `gold_doc_ids`) را در زمینه پرامپت مدل قرار دهید.
۲. با استفاده از اسکریپت `score.py` و سوییچ `--partial`، امتیازات معیارهای F1/EM را استخراج کنید (این سوییچ صرفاً برای ارزیابی زیرمجموعه‌ای از داده‌های dev تعبیه شده است):

```
python score.py runs/selfcheck/predictions.jsonl --pack course_pack_v1 --split dev --partial
```

۳. مدل باید دست‌کم در **حدود ۲۵ درصد** موارد پاسخ کاملاً منطبق (EM) تولید کرده و **قالب پاسخ** را به درستی رعایت کند (پاسخ کوتاه، عدم تولید جملات حاشیه‌ای، و استفاده از no/yes برای پرسش‌های دودویی).
۴. چنانچه مدل در این آزمون موفق نشد، **در همین مرحله** مدل یا ساختار پرامپت خود را اصلاح نمایید؛ تغییر مدل در مراحل بعدی مستلزم تکرار تمامی آزمایش‌ها خواهد بود.

تجربه ما نشان می‌دهد مدل‌های کوچک‌تر از حدود 4B عملکرد ضعیفی در این آزمون ارائه می‌دهند. در مدل‌های خانواده Qwen یا مدل‌های استدلالی، حالت تولید استدلال طولانی/تفکر (thinking) را غیرفعال کنید (مثلاً با تنظیم `enable_thinking=False`) تا پاسخ‌ها کوتاه و منطبق با قالب بمانند. اجرای کامل یک مدل حدود 7–8B روی رایانه شخصی برای کل ۳۰۰ پرسش همراه با زمینه، ممکن است چند ساعت زمان ببرد؛ بنابراین فاز توسعه و خطایابی را با نمونه‌های ۵۰ تایی پیش ببرید و سپس اجرای کامل نهایی را آغاز کنید یا از توان پردازشی Kaggle بهره بگیرید. اجرای همین سناریو بر روی کل ۳۰۰ پرسش مجموعه dev، معرف سامانه «زمینه طلایی» (**oracle-context**) در فاز ۱ خواهد بود.

۷ گزارش و ارائه شفاهی

۱.۷ ساختار گزارش هر فاز

گزارش هر فاز حداکثر در **۳ صفحه** (بدون احتساب بخش اختیاری پیوست نمونه‌ها) تنظیم می‌شود و شامل بخش‌های زیر است:

۱. **معرفی سامانه‌ها و مدل‌ها:** تشریح اجمالی هر سامانه در یک یا دو جمله، معرفی مدل‌ها، و پیوند مخزن خصوصی به همراه شناسه commit متناظر.
۲. **جدول نتایج تجربی:** گزارش مستقیم اعداد خروجی اسکریپت `score.py` روی مجموعه dev (کلی و به تفکیک دسته‌بندی نوع پرسش، به همراه فواصل اطمینان).
۳. **ادعاهای مقایسه‌ای:** مستندسازی هر ادعای برتری با تفاضل جفتی و فاصله اطمینان متناظر (`--compare`).
۴. **تحلیل خطا:** بررسی دقیق دست‌کم **۵ نمونه مشخص** از خطاها (شامل شناسه پرسش، پاسخ سامانه، پاسخ مرجع، قطعه‌متن‌های بازیابی‌شده و علت بروز خطا) همراه با دسته‌بندی انواع خطا.
۵. **تصمیمات طراحی و محدودیت‌ها:** تحلیل چرایی انتخاب رویکردها، ایده‌های آزموده‌شده که به شکست انجامیدند، و محدودیت‌های روش.
۶. **شفافیت استفاده از ابزارهای هوش مصنوعی:** توضیح صریح پیرامون ابزارهای مورد استفاده و بخش‌هایی از پروژه که با کمک آن‌ها پیش رفته است.

۲.۷ جلسه ارائه شفاهی

ارائه شفاهی **محور اصلی ارزیابی و نمره‌دهی** است: جلسه‌ای **انفرادی به مدت ۱۵ دقیقه** با دستیار آموزشی مسئول همان فاز. رایانه شخصی یا نوت‌بوک Kaggle خود را آماده داشته باشید تا سامانه بتواند روی چند پرسش نمونه به صورت زنده اجرا شود.

ساختار معمول جلسه ارزیابی شفاهی

- **اجرای زنده خط لوله:** تعداد ۳ تا ۵ پرسش انتخابی توسط دستیار آموزشی توسط خط لوله شما پردازش می‌شود؛ در این فرآیند، قطعه‌متن‌های بازیابی‌شده، زمینه ورودی مدل و پاسخ تولیدی نمایش داده خواهند شد. در صورت کندی مدل، نسخه سبک‌تری از خط لوله را برای اجرای سریع زنده آماده داشته باشید.
- **بررسی کد سازوکارهای اصلی:** بررسی خط‌به‌خط پیاده‌سازی دو سازوکار اصلی فاز و تشریح کامل منطق الگوریتمی آن‌ها.
- **پرسش‌های تحلیلی و سناریوهای تغییر:** تحلیل اینکه در صورت تغییر در یک بخش (مثلاً اعمال ماسک اسناد مستثنی پس از انتخاب k نتیجه برتر، تنظیم $b = 0$ در BM25، یا تغییر بودجه توکن‌های زمینه) چه اثری بر خروجی گذاشته می‌شد.
- **تفسیر و تحلیل نتایج:** بحث پیرامون دلایل افت یا بهبود عملکرد روی دسته‌های مختلف پرسش‌ها، مفهوم فواصل اطمینان، و تحلیل یک نمونه خطا.

دستیار آموزشی پیش از جلسه چند نمونه از پیش‌بینی‌های ارسالی شما را بررسی می‌کند و ممکن است از شما بخواهد همان موارد را در جلسه بازتولید نمایید (با تنظیم دما روی صفر، خروجی‌ها باید همسان باشند).

تسلط بر کد، شرط اساسی کسب نمره

کدهای نوشته‌شده به کمک ابزارهای هوش مصنوعی مجاز هستند، اما در صورتی که نتوانید منطق کد را در جلسه توضیح دهید یا تغییر خواسته‌شده را در آن اعمال کنید، آن بخش به عنوان کار مستقل شما پذیرفته نخواهد شد. سازوکارهای اصلی را الزاماً خودتان پیاده‌سازی کرده و بر تمام جزئیات آن مسلط باشید.

۸ فاز ۱: سنجش و بازیابی متراکم

شناسنامه و مشخصات کلیدی این فاز

هدف: ساخت ابزار سنجش استاندارد و ایجاد نخستین خطوط پایه پیش از اعمال هرگونه بهبود الگوریتمی.
سازوکارهای اصلی (دست‌نویس): (الف) پیاده‌سازی معیارهای ارزیابی بازیابی؛ (ب) بازیابی متراکم دقیق به همراه فیلتر اسناد مستثنی (document mask).
سامانه‌ها: سامانه بدون‌بازیابی (**closed-book**)، سامانه با زمینه طلایی (**oracle-context**، صرفاً روی dev) و سامانه RAG متراکم.
تحویلی: خروجی اجرای dev برای هر سه سامانه؛ خروجی اجرای hidden برای سامانه بدون‌بازیابی و متراکم؛ گزارش حداکثر ۳ صفحه‌ای؛ معرفی مدل‌ها.
دستیار مسئول: محمد امین حائری

۱.۸ سازوکار اصلی الف: معیارهای ارزیابی بازیابی

معیارهای زیر را **توسط خودتان و بدون استفاده از کتابخانه‌های آماده** برای هر پرسش و سپس میانگین آن‌ها را بر روی کل پرسش‌ها پیاده‌سازی کنید. در این روابط، G مجموعه قطعه‌متن‌های طلایی و $d_1, d_2, \ldots$ فهرست رتبه‌بندی‌شده شناسه‌های اسناد بازیابی‌شده است:

$$\text{Recall@}k = \frac{|G \cap \{d_1, \ldots, d_k\}|}{|G|}, \qquad \text{HitAll@}k = \mathbf{1}[G \subseteq \{d_1, \ldots, d_k\}],$$

$$\text{MRR} = \frac{1}{\min\{i : d_i \in G\}},$$

$$\text{DCG@}k = \sum_{i=1}^{k} \frac{\mathbf{1}[d_i \in G]}{\log_2(i+1)}, \qquad \text{nDCG@}k = \frac{\text{DCG@}k}{\sum_{i=1}^{\min(|G|,k)} \frac{1}{\log_2(i+1)}}.$$

در صورتی که هیچ قطعه‌متن طلایی بازیابی نشود، مقدار MRR برابر صفر است. بهره (gain) در محاسبه DCG به‌صورت دودویی تعریف می‌شود (هر قطعه‌متن طلایی دارای بهره ۱ است) و مقدار ایدئال (IDCG) بر روی $\min(|G|, k)$ رتبه نخست محاسبه می‌گردد. در گزارش خود در قالب یک جدول نشان دهید که خروجی پیاده‌سازی شما روی اجراهای مجموعه dev با اعداد خروجی اسکریپت `score.py` (برای معیارهای Recall@2/5/10، HitAll@2/5/10، MRR و nDCG@10) کاملاً **منطبق و یکسان** است. توجه داشته باشید که معیار HitAll@k در پرسش‌های چندمرحله‌ای اهمیت ویژه‌ای نسبت به Recall@k دارد؛ زیرا در این نوع پرسش‌ها، استدلال و پاسخ‌گویی صحیح تنها زمانی امکان‌پذیر است که **تمامی** شواهد لازم در زمینه بازیابی شده باشند.

۲.۸ سازوکار اصلی ب: بازیابی متراکم دقیق با ماسک اسناد مستثنی (Document Mask)

بردارهای تعبیه تمامی قطعه‌متن‌های پیکره را با مدل تعبیه انتخابی خود محاسبه کرده و تابعی پیاده‌سازی کنید که برای هر پرسش، شباهت کسینوسی **دقیق** (و نه تقریبی) را با تمامی قطعه‌متن‌ها به دست آورد و k نتیجه برتر را بازگرداند. این تابع باید پارامتری تحت عنوان **ماسک اسناد** (document mask یا `exclude_ids`) بپذیرد: مجموعه‌ای از شناسه‌های اسناد که باید پیش از استخراج k نتیجه برتر کنار گذاشته شوند (برای نمونه، با تنظیم امتیاز شباهت آن‌ها به $-\infty$).

```
def dense_search(query, k, exclude_ids=frozenset()) -> list[str]:  # ranked doc ids
    # 1) embed query; 2) cosine scores against ALL passage embeddings
    # 3) apply the mask (scores of excluded ids = -inf)  <-- BEFORE top-k
    # 4) return the ids of the k highest scores, best first
```

اعمال ماسک **پیش از** انتخاب k نتیجه برتر الزامی است؛ زیرا در صورت اعمال فیلتر پس از انتخاب k نتیجه، ممکن است تعداد نتایج خروجی کمتر از k شود. به الزامات فنی مدل تعبیه توجه داشته باشید: برخی مدل‌ها نیازمند پیشوند یا دستورالعمل خاص برای پرسش‌ها هستند؛ همچنین حتماً بردارهای تعبیه را پیش از محاسبه ضرب داخلی نرمال‌سازی کنید (L2 normalization). استفاده از کتابخانه‌هایی چون FAISS یا پایگاه‌داده‌های برداری در خط لوله مجاز است، اما این تابع دقیق دستی باید پیاده‌سازی شده و به عنوان مرجع مقایسه نگهداری شود.

۳.۸ سامانه‌های این فاز

۱. **بدون‌بازیابی (closed-book):** مدل زبانی بدون دسترسی به هیچ زمینه بازیابی‌شده‌ای پاسخ می‌دهد؛ در این حالت مقدار `"retrieved": []` ثبت می‌شود. این سامانه مشخص می‌کند مدل صرفاً بر پایه دانش پارامتریک خود چه میزان توانایی پاسخ‌گویی دارد.

۲. **زمینه طلایی (oracle-context):** قطعه‌متن‌های طلایی مستقیماً در زمینه ورودی مدل قرار می‌گیرند؛ مقدار فیلد `retrieved` معادل با `gold_doc_ids` خواهد بود. این سامانه کران بالای تقریبی عملکرد مرحله تولید مدل زبانی را نمایان می‌سازد. اجرای این سامانه **صرفاً بر روی مجموعه dev** مقدور است؛ زیرا برچسب‌های طلایی مجموعه hidden منتشر نمی‌شوند.

۳. **RAG متراکم (Dense RAG):** تعداد k قطعه‌متن برترِ بازیابی‌شده توسط بازیاب متراکم (با مقدار k و ساختار پرامپت انتخابی شما) در زمینه مدل قرار می‌گیرند؛ فیلد `retrieved` باید دست‌کم شامل ۱۰ شناسه باشد.

مقایسه این سه سامانه تصویر تحلیلی روشنی به دست می‌دهد: فاصله عملکرد سامانه بدون‌بازیابی تا RAG متراکم نشان‌دهنده ارزش افزوده و سهم بازیابی است؛ در حالی که فاصله RAG متراکم تا سامانه زمینه طلایی، پتانسیل و ظرفیت ارتقای بازیاب را معین می‌کند. نتایج را به تفکیک دسته‌بندی پرسش‌ها نیز تحلیل کنید. توجه نمایید که معیارهای ارزیابی بازیابی برای سامانه زمینه طلایی و سامانه بدون‌بازیابی فاقد معنا هستند (یکی عیناً منطبق بر شواهد طلایی است و دیگری فاقد سند بازیابی‌شده است)؛ بنابراین نیازی به گزارش معیارهای بازیابی برای این دو سامانه نیست.

پرسش‌های تحویلی و مستندسازی این فاز

- مدل‌های اعلام‌شده (LLM، تعبیه، بازرتبه‌بندی‌کننده) و نتیجه خودآزمایی مدل (بخش ۶).
- خروجی اجراهای dev: بدون‌بازیابی، زمینه طلایی، متراکم؛ اجراهای hidden: بدون‌بازیابی، متراکم (هر یک همراه با `run_config.yaml`).
- جدول مقایسه و تطابق معیارهای دست‌نویس با خروجی `score.py`.
- جدول نتایج تجربی سه سامانه (کلی و به تفکیک دسته‌بندی) و دست‌کم یک تحلیل مقایسه جفتی با `--compare` (مانند مقایسه سامانه متراکم در برابر بدون‌بازیابی).
- تحلیل خطا: میزان معیار HitAll@10 در پرسش‌های دسته bridge_comparison چقدر است و علل ریشه‌ای آن چیست؟

۹ فاز ۲: بازیابی ترکیبی (هایبرید) و پاسخ‌های مستند

شناسنامه و مشخصات کلیدی این فاز

هدف: اضافه کردن بازیابی متنی (Lexical / BM25) و تلفیق آن با بازیابی معنایی (Dense) به صورت هایبرید (Hybrid RAG)؛ تولید پاسخ‌های مستند به شواهد به همراه ارجاع (Citation) و پیاده‌سازی مکانیزم امتناع از پاسخ (Abstention).

سازوکارهای اصلی (دست‌نویس): (الف) پیاده‌سازی الگوریتم BM25 و ترکیب رتبه‌ها با روش RRF؛ (ب) بسته‌بندی کانتکست بر مبنای سقف بودجه توکن (Context Packing)، همراه با استخراج ارجاعات و تشخیص امتناع از پاسخ.

سامانه‌ها: سامانه Hybrid RAG (با قابلیت ارجاع و امتناع)، آزمایش مقایسه واحد بازیابی (Retrieval Unit Granularity)، آزمون ارزیابی امتناع با ماسک کردن اسناد طلایی (Gold-Masked Abstention)؛ استفاده اختیاری از Reranker.

تحویلی: خروجی‌های مجموعه dev برای همه سامانه‌ها؛ خروجی مجموعه hidden برای سامانه هایبرید؛ گزارش فنی حداکثر ۳ صفحه‌ای.

دستیار مسئول: پوریا صانعی

۱.۹ سازوکار اصلی الف: الگوریتم BM25 و ادغام رتبه‌ها با RRF

در این بخش باید الگوریتم BM25 را خودتان از پایه پیاده‌سازی کنید (شامل مراحل ساده پیش‌پردازش و توکنایز کردن، تبدیل حروف بزرگ به کوچک، و حذف کلمات توقف (stop words) در صورت نیاز). فرمول امتیازدهی به یک سند یا پسج d برای پرسش q به صورت زیر است:

$$\mathrm{BM25}(q,d) = \sum_{t \in q} \mathrm{idf}(t) \cdot \frac{f(t,d)\,(k_1+1)}{f(t,d) + k_1\left(1 - b + b\,\frac{|d|}{\mathrm{avgdl}}\right)}, \qquad \mathrm{idf}(t) = \ln\left(1 + \frac{N - \mathrm{df}(t) + 0.5}{\mathrm{df}(t) + 0.5}\right)$$

که در آن:

- $f(t,d)$ بسامد و تعداد تکرار کلمه t در سند d است (Term Frequency).
- $|d|$ طول سند d بر حسب تعداد توکن و avgdl میانگین طول تمام اسناد در کل پیکره است.
- N تعداد کل اسناد و $\mathrm{df}(t)$ تعداد اسنادی است که شامل کلمه t هستند (Document Frequency).
- مقادیر پیش‌فرض پارامترها را برابر $k_1 = 1.5$ و $b = 0.75$ قرار دهید.

از نسخه فوق برای idf استفاده کنید تا مقدار آن همیشه مثبت بماند. همچنین در مواجهه با کلماتی از پرسش که در واژگان وجود ندارند (OOV) یا پرسش‌های بدون تطابق، به جای بروز خطا (crash)، امتیاز صفر در نظر بگیرید. برای اطمینان از درستی پیاده‌سازی می‌توانید خروجی خود را با کتابخانه rank_bm25 بسنجید، اما کد نهایی باید دست‌نویس خودتان باشد. قابلیت فیلتر اسناد مستثنی (همان ماسک کردن اسناد در فاز ۱) باید در این بازیاب نیز پشتیبانی شود.

سپس خروجی رتبه‌بندی بازیاب متراکم (فاز ۱) و بازیاب BM25 را با روش **ادغام معکوس رتبه‌ها** (**Reciprocal Rank Fusion - RRF**) ترکیب کنید:

$$\mathrm{RRF}(d) = \sum_{r \in \{\text{dense},\,\text{bm25}\}} \frac{1}{60 + \mathrm{rank}_r(d)}$$

در این رابطه، رتبه‌بندی (rank) از ۱ شروع می‌شود (نه صفر). اگر سندی در یکی از دو فهرست حضور نداشته باشد، سهم امتیازی از آن منبع دریافت نخواهد کرد. برتری کلیدی RRF این است که صرفاً بر مبنای جایگاه و رتبه اسناد کار می‌کند و نیازی به نرمال‌سازی یا هم‌مقیاس کردن نمرات نامتجانس BM25 و شباهت کسینوسی ندارد. در صورت برابری امتیاز، ترتیب تساوی را به شکل قطعی (deterministic) مدیریت کنید (مثلاً بر اساس شناسه سند).

۲.۹ سازوکار اصلی ب: بسته‌بندی کانتکست، استناددهی و امتناع از پاسخ

- **بسته‌بندی کانتکست با سقف بودجه توکن (Context Packing):** در سامانه‌های واقعی به دلیل محدودیت پنجره کانتکست و زمان پردازش مدل زبانی، نمی‌توان تمام اسناد را به پرامپت فرستاد. تابعی بنویسید که اسناد بازیابی‌شده را به ترتیب اولویت و رتبه، تا سقف مشخصی از تعداد توکن (مثلاً ۱۸۰۰ توکن) در کانتکست پرامپت قرار دهد و هر پسج را با شناسه معتبر برچسب‌گذاری کند. **نکته کلیدی:** شمارش توکن‌ها باید حتماً توسط **توکنایزر خودِ مدل زبانی (LLM)** و بر روی کل بلوک متنی ورودی (شامل شناسه‌ها، عنوان‌ها و جداکننده‌ها) محاسبه شود. اگر افزودن یک پسج باعث عبور از سقف توکن شد، یک سیاست قطعی در پیش گیرید (حذف کامل آن پسج یا برش متن آن). مقدار سقف بودجه را در فایل `run_config.yaml` مشخص کنید.
- **استناد و امتناع از پاسخ (Citation & Abstention):** پرامپت را به شکلی طراحی کنید که مدل علاوه بر پاسخ کوتاه، شناسه‌های اسناد پشتیبان را در فیلد `cited` ثبت کند، و چنانچه اطلاعات کافی در اسناد موجود نبود، صریحاً با تنظیم `abstained: true` و قرار دادن پاسخ خالی (`""`) از پاسخ دادن امتناع نماید. کد شما باید خروجی مدل را **پارس و اعتبارسنجی کند** و در مواجهه با خروجی‌های دارای توضیحات اضافی، بلوک‌های تفکر (thinking) یا قالب‌های نامتعارف دچار خطا نشود. همچنین کنترل کنید که شناسه‌های ثبت‌شده در `cited` ، دقیقاً همان شناسه‌هایی باشند که در کانتکست ورودی حضور داشته‌اند.

۳.۹ آزمایش‌های تجربی این فاز

۱. **سامانه Hybrid RAG:** ترکیب بازیاب متراکم فاز ۱ و بازیاب BM25 با ادغام RRF، همراه با کانتکست بسته‌بندی‌شده، ارجاع‌دهی و قابلیت امتناع. نتایج را با دستور `--compare` در اسکریپت `score.py` در برابر سامانه متراکم فاز ۱ مقایسه نمایید (با حفظ شرایط منصفانه مقایسه، مانند سقف توکن و پارامترهای یکسان).

۲. **بررسی اندازه واحد بازیابی (Granularity / Retrieval Unit):** بازیابی در سطح پسج کامل را با واحدهای دیگر مقایسه کنید؛ مانند بازیابی در سطح جمله (استفاده از فیلد `sentences` و سپس تجمیع امتیازها روی سطح سند با max-pooling)، یا افزودن عنوان مقاله (title) به ابتدای متن پسج (جهت شفاف‌سازی مرجع ضمیرها). در خروجی نهایی همواره باید شناسه پسج اصلی (`doc_id`) ثبت گردد.

۳. **بازرتبه‌بندی‌کننده (Reranker - اختیاری):** در صورتی که در فاز ۱ استفاده از reranker را اعلام کرده بودید، اثر آن را بر روی چند ده نتیجه برتر ارزیابی نمایید.

۴. **آزمون امتناع با ماسک اسناد طلایی (Gold-Masked Abstention):** روی داده‌های dev، با استفاده از مکانیزم ماسک اسناد، پسج‌های طلایی هر پرسش را پیش از بازیابی حذف کنید (در هر دو بازیاب متراکم و BM25). در این وضعیت شواهد پاسخ در دسترس نخواهد بود و سامانه مطلوب باید به جای ساخت پاسخی توهم‌آمیز، **از پاسخ دادن امتناع کند**. نرخ امتناع (abstained)، پاسخ غلط (hallucination) و پاسخ‌های درست تصادفی (lucky-correct) را مقایسه و تحلیل نمایید.

پرسش‌های تحویلی و مستندسازی این فاز

- خروجی‌های مجموعه dev: سامانه هایبرید، آزمایش واحد بازیابی، سامانه هایبرید با ماسک طلایی (و نسخه دارای reranker در صورت استفاده)؛ خروجی مجموعه hidden: سامانه هایبرید نهایی.
- تحلیل مقایسه جفتی (paired comparison) سامانه هایبرید با سامانه متراکم فاز ۱ به تفکیک نوع پرسش؛ کدام دسته‌ها بیشترین بهبود را نشان داده‌اند و چرا؟
- ارزیابی دقت و بازخوانی ارجاعات (cite_precision و cite_recall) و تحلیل تجربی نتایج آزمون امتناع در مواجهه با اسناد ماسک‌شده.

آ پیوست اختیاری: امتیاز مهندسی نرم‌افزار و ردیابی آزمایش‌ها

هیچ چارچوب یا ابزار مهندسی خاصی اجباری نیست؛ با این وجود، رعایت اصول مهندسی نرم‌افزار و شیوه‌های حرفه‌ای توسعه می‌تواند مشمول **امتیاز تشویقی** در ارزیابی نهایی شود. این موارد **صرفاً در جلسه ارائه شفاهی و به‌صورت بسیار مختصر** (۱ تا ۲ دقیقه) بررسی می‌گردند و نیازی به ارسال مستندات جداگانه برای آن‌ها نیست.

۱.آ پیکربندی ساختاریافته، آزمون خودکار و یکپارچه‌سازی مداوم

- **مدیریت پیکربندی (Configuration Management):** نگهداری تمامی تنظیمات و فراپارامترهای هر سامانه در فایل‌های ساختاریافته (نظیر YAML) به جای مقادیر ثابت در متن کد؛ به شکلی که فایل‌های `run_config.yaml` مستقیماً از روی آن‌ها تولید شوند.
- **آزمون‌های واحد (Unit Tests):** طراحی تست با ابزارهایی مانند pytest: ارزیابی توابع معیارهای دست‌نویس با مثال‌های ساده و دستی، اطمینان از اینکه ماسک اسناد مستثنی قطعه‌متن‌های تعیین‌شده را فیلتر می‌کند، راستی‌آزمایی تابع ادغام RRF بر روی دو فهرست نمونه، و اطمینان از تاب‌آوری پردازشگر خروجی مدل (parser) در مواجهه با خطاهای ساختاری.
- **یکپارچه‌سازی مداوم (CI):** تعریف یک گردش‌کار سبک در GitHub Actions به نحوی که با هر push، آزمون‌های واحد کد به‌صورت خودکار اجرا شوند (بدون نیاز به بارگذاری مدل‌های زبانی یا داده‌های حجیم).

۲.آ ردیابی آزمایش‌ها با MLflow یا W&B

با افزایش تعداد اجراها و تنظیمات مختلف، ابزارهای ردیابی آزمایش (Experiment Tracking) به شما کمک می‌کنند تا منشأ دقیق هر نتیجه تجربی و تنظیمات متناظر با آن را به سادگی رهگیری نمایید.

- **MLflow:** نسخه این ابزار را در فایل پیش‌نیازها تثبیت کنید (مانند `mlflow==<version>`) تا بر روی سیستم محلی و بستر Kaggle یکسان باشد. به جای ذخیره محلی در پوشه‌های پراکنده، از یک پایگاه‌داده سبک SQLite استفاده کنید:

```
import mlflow
mlflow.set_tracking_uri("sqlite:///mlflow.db")
mlflow.set_experiment("phase2")
with mlflow.start_run(run_name="hybrid_rrf"):
    mlflow.log_params({"k": 10, "rrf_k": 60, "max_context_tokens": 1800})
    mlflow.log_metrics({"em": 0.41, "hitall_10": 0.55})
    mlflow.log_artifact("runs/dev/hybrid/run_config.yaml")

# then, in a terminal:
mlflow ui --backend-store-uri sqlite:///mlflow.db
```

- **Weights & Biases:** این بستر برای مصارف دانشگاهی رایگان است و نیازی به مدیریت فایل محلی ندارد.
- در جلسه ارائه شفاهی، کافی است داشبورد ردیابی خود را نمایش دهید و ارتباط یکی از اعداد گزارش را با اجرای متناظر مشخص سازید.

راهنمای فنی اجرای MLflow بر روی بستر Kaggle (ذخیره‌سازی پایگاه داده SQLite میان نشست‌های کاری) در مستند راهنمای Kaggle تشریح شده است.